INFORME TÉCNICO - SESIÓN 10

Autenticación JWT, CRUD protegido y Oracle en contenedores para PharmaMobile_Practica

ESTUDIANTE	Guillen Mendoza Alexander	**DOCENTE**	David Reyna
ASIGNATURA	Desarrollo de Aplicaciones Móviles	**FECHA**	13 de octubre de 2026
PROYECTO	PharmaMobile_Practica + PharmaSoft	**ENTREGA**	GitHub y PDF

1. Resumen ejecutivo

Se integró una sesión JWT de PharmaSoft en la app Kotlin Multiplatform. La solución incluye login, recuperación y cierre de sesión, token cifrado por plataforma, autenticación automática en Ktor, tratamiento distinto de 401 y 403, y secciones REST para productos y clientes. En el servidor se levantaron Oracle Free y Spring Boot mediante Docker Compose, con esquema y datos semilla aplicados por Flyway.

Resultado comprobado: Oracle saludable, cuatro migraciones aplicadas, autenticación y autorización HTTP correctas, CRUD real de un producto temporal (201/200/204), pruebas Maven completas sin fallos y 28 pruebas móviles de host correctas. La ejecución visual de la app contra el backend y la ejecución iOS aún requieren evidencia adicional.

2. Objetivo y contexto

La Guía Práctica N.º 10 solicita integrar el login JWT, adjuntar el token con el plugin Auth de Ktor, protegerlo en Android e iOS, ocultar el token en los registros y distinguir sesión expirada de falta de permisos. Este informe corresponde a la implementación realizada sobre la rama móvil que ya contenía el trabajo de sesiones anteriores. Se conservaron el inventario local demostrativo y el catálogo público de Platzi; Pedidos sigue siendo una demostración, no un CRUD REST.

Los insumos recibidos fueron la guía PDF, una colección Postman de 63 solicitudes de seguridad y un RAR con migraciones V2/V3. El RAR no era una base móvil; su V2 contenía una línea SQL inválida, por lo que se empleó la migración corregida del repositorio PharmaSoft. No se copiaron credenciales ni secretos al código fuente.

3. Arquitectura de la solución

CAPA	RESPONSABILIDAD IMPLEMENTADA
Dominio	Modelos de sesión y errores, contratos de repositorio y casos de uso; sin Ktor, Android ni UIKit.
Datos comunes	DTO, cliente Ktor, Auth Bearer, traducción de errores y repositorios REST de autenticación, productos y clientes.
Plataformas	AlmacenSeguroAndroid con Android Keystore/EncryptedSharedPreferences; AlmacenSeguroIos con Keychain; URL base por expect/actual.
Presentación	ViewModels, LoginScreen, pantallas de PharmaSoft, estados de carga/error, navegación y acciones visibles según permisos.
Servidor	Spring Security/JWT, controladores REST, Oracle Free y migraciones Flyway V1-V4.

4. Funcionalidad móvil implementada

Inicio y recuperación: si no existe token se presenta el login. Con token guardado, la app consulta /auth/me para recuperar identidad y permisos. Un fallo temporal de red permite reintentar sin borrar automáticamente la credencial.

Login: POST /api/v1/auth/login envía usuario y contraseña; el repositorio guarda el JWT en AlmacenSeguro, limpia la caché del proveedor Bearer y descarta la contraseña del estado de interfaz. El DTO de /auth/me se ajustó al campo real empleadoNombre.

Solicitudes protegidas: un cliente Ktor exclusivo de PharmaSoft instala ContentNegotiation, DefaultRequest, Auth Bearer, HttpTimeout y Logging. El token se envía desde la primera solicitud a /api/v1/ excepto al login. No se escribe Authorization manualmente en los composables. El cliente público de Platzi permanece separado.

Seguridad y errores: el logging de depuración usa cabeceras y sanea Authorization como ***; en otras compilaciones está apagado. Un 401 protegido borra la sesión y vuelve al login; un 401 del login significa credenciales inválidas. Un 403 muestra falta de permiso sin cerrar la sesión. El cierre voluntario elimina el token y limpia la caché Bearer.

Permisos y CRUD: CATALOGO_LEER/ESCRIBIR y CLIENTES_LEER/ESCRIBIR determinan qué secciones y botones muestra la interfaz. Las pantallas REST permiten listar, registrar, actualizar y eliminar productos y clientes. La restricción visual no sustituye la autorización del backend.

5. Implementación del servidor y Oracle

Docker Compose ejecuta una imagen oficial de Oracle Free y el backend Spring Boot. Oracle escucha en el puerto local 1522; la API, en el 8080. Los puertos se publican solo en 127.0.0.1. El volumen oracle_data conserva los archivos de la base entre reinicios. Un script idempotente crea PHARMADB al arranque si no existe y aplica sus permisos; se fijó el formato LF del script para evitar errores de Bash en Windows.

El perfil container configura JDBC, Hibernate y Flyway. V1 crea el esquema inicial; V2 agrega empleados, usuarios y roles; V3 siembra usuarios de desarrollo con hashes BCrypt; V4 agrega tres categorías. Se corrigió la configuración CORS requerida por Spring Security. Las contraseñas y el secreto JWT están en un archivo .env ignorado por Git, nunca en el PDF ni en el repositorio.

MIGRACIÓN	RESULTADO COMPROBADO
V1	Esquema inicial del negocio aplicado por Flyway.
V2	Tablas y relaciones de seguridad aplicadas con la versión SQL corregida.
V3	Usuarios y roles de desarrollo disponibles para pruebas JWT.
V4	Tres categorías recuperadas desde la API real.

6. Verificación ejecutada

PRUEBA	EVIDENCIA O RESPUESTA	ESTADO
Oracle / PHARMADB	Contenedor saludable y conexión SQL real	Correcto
Flyway	Cuatro migraciones conservadas tras recrear Oracle	Correcto
GET /api/health	200	Correcto
GET productos sin token	401	Correcto
Login admin y /auth/me	Token emitido; /auth/me devuelve admin	Correcto
GET productos con admin	200	Correcto
GET categorías	Tres registros	Correcto
GET clientes con almacen	403; la sesión sigue autorizada para catálogo	Correcto
Producto temporal	POST 201, GET 200, DELETE 204	Correcto
Pruebas Maven	mvnw.cmd test contra Oracle	Sin fallos
Pruebas y compilación móvil	28 pruebas host; APK debug y Kotlin iOS compilados	Correcto

La prueba del producto usó un registro temporal que se eliminó al terminar. Tras recrear Oracle sin borrar el volumen, siguieron presentes las migraciones, las tres categorías y el login. El flujo real fue comprobado por HTTP desde el equipo anfitrión; no equivale a haberlo recorrido manualmente en la pantalla móvil.

7. Criterios de la guía: estado honesto

CRITERIO	ESTADO
1. Login JWT	Implementado; verificado por HTTP y pruebas móviles
2. Almacén por plataforma	Implementado; compilado, sin prueba visual iOS
3-4. Auth y envío inicial	Implementado; cubierto con MockEngine
5. Log saneado	Implementado y probado; falta captura de Logcat
6. Persistencia tras reapertura	Prueba automatizada; falta ciclo manual de app
7-9. 401, 403 y permisos UI	Probados con mocks; 401/403 reales del servidor comprobados
10. Android e iOS en ejecución	Pendiente de evidencia funcional en ambos

8. Publicación y reproducibilidad

La aplicación se publicó en master y en feature/auth-jwt-guillen del repositorio del estudiante. El backend se publicó en backend/oracle-jwt del mismo repositorio, como rama independiente con sus propios archivos e historial. No debe fusionarse con master como si fuera una característica de la app: es un proyecto distinto. El remoto original del backend, perteneciente al docente, no se modificó.

CONTENIDO	RAMA Y REVISIÓN PUBLICADA
Aplicación	master y feature/auth-jwt-guillen - 5be2535
Servidor	backend/oracle-jwt - a88a22f

Repositorio: github.com/alexgm-A11/PharmaMobile_Practica

Documentación técnica: README.md y docs/S10_AUTENTICACION_JWT.md en master; README.md del backend en backend/oracle-jwt.

9. Limitaciones y siguientes verificaciones

Postman: se analizaron las 63 solicitudes, pero no se ejecutó la colección completa. Sí se verificaron directamente los casos fundamentales de 401, 200 y 403. Falta guardar capturas exigidas por la guía.

Android: el APK compiló y se había instalado/abierto en Pixel 6 sin fallo de arranque. Durante la verificación final no había emulador conectado; faltan el recorrido real de login, CRUD, reapertura con sesión guardada, 403 visible y captura del log con Authorization saneada.

iOS: compiló el código Kotlin compartido para el simulador, pero Windows no ejecuta iosApp ni demuestra el comportamiento del Keychain. La prueba y capturas iOS requieren macOS y Xcode.

Producción: sustituir usuarios semilla y contraseñas de desarrollo, utilizar HTTPS y revisar la estrategia de almacenamiento Android, pues EncryptedSharedPreferences está obsoleta. El tráfico HTTP claro se permite únicamente en la compilación Android debug.

10. Conclusión

La integración JWT y la infraestructura Oracle quedaron implementadas y verificadas en las capas correspondientes. El backend inicia, migra y conserva datos; el acceso protegido y un ciclo real de CRUD responden correctamente. La app dispone del flujo de sesión y CRUD autenticado, con pruebas automatizadas satisfactorias. Para presentar el criterio de funcionamiento en ambas plataformas como completo aún se necesitan las ejecuciones y capturas manuales indicadas.

Fuentes de alcance: Guía Práctica de Laboratorio N.º 10 (13/10/2026), colección Postman de seguridad y migraciones recibidas; código y documentación de las revisiones Git indicadas. Los resultados de prueba corresponden a la verificación local del 13/10/2026.